llmctl docs/user-manual

User Manual

Revision: 1 **Last modified:** 2026-09-15T00:00:00Z

Complete command reference for `bin/llmctl`. Every command below is described from the real source (`bin/llmctl`'s `main()` case statement plus the `lib/*.sh` functions each branch calls) — nothing here is guessed or inferred from the `--help` text alone (Constitution §11.4.6). For a guided first run see `docs/quickstart.md`; for wiring a coding agent against a running profile see `docs/integrations.md`.

Conventions used below

- `llmctl` is invoked as `./bin/llmctl <command> [args]` from the repository root, or as `llmctl` if the `bin/` directory is on `PATH`.
- All ports, profile names, and defaults below are taken verbatim from `models/catalog.json` and cross-checked against `docs/integrations.md`'s port table.
- Environment variables that change command behavior are documented once in Environment variables at the end of this file rather than repeated under every command.
- "Real evidence" callouts cite an actual assertion from the test suite (`tests/test_cli.sh`, `tests/test_scheduler.sh`) so the documented output shape is provably the real one, not an invented example.

Command index

Command	Purpose
`setup`	doctor → build engines → hardware plan, in one shot
`doctor`	environment self-diagnosis
`hw`	probe hardware (human or JSON)
`plan`	which catalog profiles fit this host
`models list`	list catalog profiles
`models download`	verified model download
`models verify`	re-verify a downloaded profile's check-sums
`validate`, `vision validate`, `validate-all`, `require-validated`	deterministic known-answer validation of a served model, and the consumer check
`build`	build the inference engines from source (`onnx` has no build step)
`install`	install/refresh persistent-service unit templates
`enable`	enable a profile at login + start it now
`disable`	disable autostart + stop
`start`	co-resident start (footprint-checked)
`stop`	stop one, several, or all running profiles
`restart`	restart a service
`switch`	stop everything, start exactly one profile
`auto`	

Command	Purpose
	best-fitting set for a capability, with LRU eviction
`decide`	typed decisions (noul/choice/score) against local decision models
`status`	running services, reservations, crash-loop state
`logs`	tail a service log
`cluster` , `tenant` , `apikey`	opt-in cluster client commands (need `llmctld` and an HTTP/3-capable curl)
`version`	print the llmctl version
`help`	print usage

---

setup

```
llmctl setup
```

Runs the full bring-up sequence in order (`cmd_setup` in `bin/llmctl`):

1. `doctor_run` — aborts with `die "doctor reported failures; fix them and re-run 'llmctl setup'"` if any check `FAIL` s.
2. `engine_build all` — builds both `llama.cpp` and `colibri` from the pinned `submodules/` git submodules.
3. Prints the hardware plan (`hw_probe_json | catalog_plan_json | catalog_plan_human`) — the same output as `llmctl plan` .

Ends with `info "setup complete. Next: llmctl models download <profile>, then llmctl start <profile>"` .

Takes no arguments. See `docs/quickstart.md` §1 for the expected full transcript on a fresh clone.

doctor

```
llmctl doctor
```

Environment self-diagnosis (`doctor_run` in `lib/doctor.sh`). Every check prints `PASS` , `WARN` , or `FAIL` with real evidence inline — never a bare claim. Checks performed, in order:

- OS + architecture detection.
- Required commands: `bash` , `curl` , `git` , `python3` (`FAIL` if missing).
- Optional commands: `cmake` (needed for `llmctl build llama`), `make` (needed for `llmctl build colibri`), a C compiler (`gcc` / `clang` / `cc`).
- A sha256 tool (`sha256sum` , `shasum` , or `openssl`) — `FAIL` if none, since downloads could not be verified.
- Whether `submodules/llama.cpp` and `submodules/colibri` are initialized git submodules, printing the short commit hash when they are.
- Catalog validity (`models/catalog.json` parses and has profiles).
- Whether the state directory is writable.
- GPU tooling: `nvidia-smi` , Apple Silicon unified memory, or `rocm-smi` (informational only — `WARN` , never `FAIL` , on CPU-only hosts).
- Service backend availability: `systemd --user` session on Linux, `launchctl` on macOS.
- Whether `llama-server` has already been built.
- Cluster-mode tooling (informational, never blocking single-host use): whether `curl` supports HTTP/3, and whether a Go toolchain is present to build the opt-in `llmctld` daemon.

Exit code is non-zero exactly when `DOCTOR_FAILS > 0` ; the final line prints `doctor: <N> failure(s), <N> warning(s)` .

hw [--json]

```
llmctl hw           # human-readable
llmctl hw --json    # machine-readable
llmctl probe        # alias for hw
```

Probes CPU, RAM, GPU/VRAM, and storage on the host (`hw_probe_human` / `hw_probe_json` in `lib/hardware.sh` , called directly from `main()` 's `hw|probe` branch). `--json` prints the same probe as a JSON document consumed internally by `plan` , `start` , `auto` , and `enable` .

Real evidence (from `tests/test_cli.sh` , against the fixture hardware doc): human output includes lines such as `AMD Ryzen 7 2700X Eight-Core Processor` and `NVIDIA GeForce RTX 3060` — i.e. the actual detected CPU model string and GPU name, not a generic label.

plan [--json]

```
llmctl plan           # human-readable
llmctl plan --json    # machine-readable
```

Pipes the live hardware probe through the catalog planner (`hw_probe_json | catalog_plan_json | catalog_plan_human` , or without the final stage for `--json`). Reports:

- The host's classified tier (`below-minimum` , `baseline` , `workstation` , or `datacenter` — see `catalog_classify_tier` in `lib/catalog.sh`).
- RAM/VRAM/storage budgets (available RAM minus 4 GiB headroom; total VRAM minus 15% headroom).
- Per-profile verdict: `FITS` (recommended), `GATED` (host tier too low for the profile's `min_tier`), or `NO-FIT` (footprint exceeds budget), plus mode (`gpu` / `cpu` / `colibri`), RAM/VRAM need, context size, and GPU-layer count.

- Co-residency groups: sets of profiles whose combined RAM+VRAM+storage footprint fits at the same time, computed by greedy bin-packing in port order.

Real evidence (`tests/test_cli.sh`): against the baseline fixture, output contains `Host tier: baseline` and `group 1: fast, coder, vision` — the `fast` (8080), `coder` (8081), and `vision` (8082) profiles co-resident in one group.

models list

```
llmctl models list
```

Lists every profile in `models/catalog.json` (`cmd_models_list` in `bin/llmctl`), one row per profile: `profile` , `port` , `engine` (`llama` / `colibri`), `size GiB` (rounded up from the sum of the profile's file sizes), `min-tier` , and `capabilities` (space-separated, e.g. `chat` , `coder` , `vision`). Current catalog profiles: `fast` (8080), `coder` (8081), `vision` (8082), `vision-pro` (8083), `moe-fast` (8084), `small` (8085), `ws-dense-32b` (8086), `ws-moe-30b` (8087), `colibri-glm` (8090), `colibri-qwen36` (8091).

models download <profile>

```
llmctl models download coder
```

Verified download for one catalog profile (`download_profile` in `lib/download.sh`). For each file the catalog lists:

1. Skips the download if the file already exists on disk **and** its sha256 matches the catalog (or a live-fetched Hugging Face checksum when the catalog entry has none).
2. Otherwise downloads via `curl -L --continue-at -` into a `.part` file (resumable; falls back to a full re-download from byte 0 if the server answers curl exit 33, "no Range support").

3. Verifies size and sha256 **before** the atomic rename to the final path — a checksum mismatch never lands at the destination.
4. Runs a post-download validation appropriate to the engine:
    - `llama` (GGUF) profiles: a real smoke test — starts `llama-server` against the downloaded model on a free `127.0.0.1` port (`LLMCTL_SMOKE_PORT` , default `auto`) (default `18090`), waits for `/health` , then sends the deterministic prompt `"Reply with exactly: OK"` via `/v1/chat/completions` and requires the response to contain `OK` . Skipped with a warning (not a failure) if `llama-server` has not been built yet.
    - `colibri` profiles: `coli doctor` if the `coli` launcher is installed, else a structural check (at least one non-empty `.safetensors` shard plus a `config*.json` file).
5. Appends every command, exit code, and raw output to `$LLMCTL_VERIFY_DIR/<profile>.log` as captured evidence.

Example: `llmctl models download fast` downloads Llama 3.1 8B Instruct Q4_K_M (~4.9 GB, per `models/catalog.json`) and smoke-tests it. Any mismatch anywhere in this chain is a hard failure (non-zero exit).

models verify <profile>

```
llmctl models verify coder
```

Re-verifies the checksums of an already-downloaded profile (`verify_profile` in `lib/download.sh`) without re-downloading or re-running the smoke test. Fails with `die "profile

not downloaded (no

)"` if the profile's model directory does not exist, and fails on the first file whose size or sha256 no longer matches the catalog.

After the checksums it also runs the validation-suite gate (`models_verify_gate` in `lib/validate.sh`): the profile must name an existing validation suite. A profile that names no validation suite (the

typed-decision profiles) prints a warning that it is **NOT validated** and the command still exits 0, because the exit code is the checksum verdict; with `--live` the same profile exits 2 (cannot validate). When the profile is running (or with `--live`) its suite is also run against the live endpoint. See validation.

validate | vision validate | validate-all | require-validated

```
llmctl validate <profile> [--host URL] [--json] [--keep]
llmctl vision validate [--profile vision|vision-pro] [--host URL] [--json] [--keep]
llmctl validate-all
llmctl require-validated <profile>
```

`validate` runs every validation suite the catalog names for the profile (`validation.suites` in `models/catalog.json`) against its running llama-server; a downloaded but idle profile is started through the scheduler first (budget-checked) and stopped again unless `--keep`. `vision validate` runs only the golden-screenshot suite (`validation/suites/vision-golden-v1/`): 9 screenshots with known content (large text, shape counts, login form, list, table, dialog, dark theme, small text, colour order) plus 2 golden-bad controls (blank image, wrong-content image). Each answer is printed next to its verdict, e.g.

```
[PASS] 02_shapes_count/red_circles (0.88s, finish=stop)
       answer: "3"
[PASS] control 90_bad_blank vs 01_text_large/read: golden checks correctly FAIL on t
VERDICT: PASS pass=38 fail=0 undetermined=0
```

Exit codes: `0` every check passed, `1` any wrong answer, `2` could not determine (not downloaded, not served, endpoint down, wrong model served, memory pressure, no suite). `--host URL` validates an explicit endpoint; the result is evidence only and is never recorded as local state.

`validate-all` validates every running profile (exit 2 when nothing runs) and then lists EVERY catalog profile without a signed, unexpired PASS record as `NOT VALIDATED` (profiles whose suite never ran against their

real model say so). `require-validated <profile>` exits 0 only when the profile has a fresh, signed passing record on this machine that matches the sha256 of the model files on disk (and their catalog pins), the engine build, the launch flags and the suite content, younger than `LLMCTL_VALIDATION_MAX_AGE` (whole seconds, default 7 days). `llmctl endpoint <profile>` prints the URL only for a running, published, validated profile - the call a project or service makes before it uses a model.

Enforced, strict by default. `start`, `switch`, `auto`, `enable`, `restart` and `decide scale` never serve a profile without a fresh record: it is started unpublished (127.0.0.1, a private port, no registry row), validated, and published only on PASS; FAIL **and** UNDETERMINED stop it. Once `llmctl install` has been run with this version, a systemd unit without a fresh record is skipped at boot/restart (exit 77); units installed by an older llmctl keep their old behaviour until then - validate the profiles you serve before you run it (upgrading installed units). The opt-out `LLMCTL_VALIDATE_STRICT=0` (serve an UNDETERMINED profile marked UNVALIDATED) is refused for consumer-facing profiles - every shipped profile is one. `LLMCTL_VALIDATE_REPEAT` (default 3) asks every probe that many times; `1` is a quick mode that is never recorded. Details, judge rules and enforcement: validation.

build [all|llama|colibri|onnx]

```
llmctl build all        # llama + colibri + onnx + decide
llmctl build llama      # llama.cpp only
llmctl build colibri    # colibri only
llmctl build onnx       # the hash-locked private venv of the onnx engine
llmctl build decide     # the Go decision binary build/llmctl-decide (needs Go >= 1.
```

A **target is required** (3.1.0): a bare `llmctl build` prints `usage: llmctl build <llama|colibri|onnx|decide|all>` and exits 2 (it used to start compiling everything).

Builds the inference engines from the pinned `submodules/llama.cpp` and `submodules/colibri` git submodules (`engine_build` in `lib/engine.sh`), initializing them first via `git submodule update --init --recursive` if either is not yet checked out.

- `llama` : configures with `cmake` and builds the `llama-server` target via `cmake --build` . Backend is auto-detected (`engine_detect_backend`): macOS → Metal (on by default upstream, nothing added); `nvcc` on `PATH` or `/usr/local/cuda/bin/nvcc` present → `-DGGML_CUDA=ON` ; `rocm-smi` or `/opt/rocm` present → `-DGGML_HIP=ON` ; otherwise a CPU build with `-DGGML_NATIVE=ON` (host-native optimization). Job count comes from `llmctl_nproc` . After building it runs `llama-server --version` and dies if that fails, so a "successful" build always means a verified-runnable binary. The llama.cpp pin is tag `b11379` (it was `b10969`); the build is shared-library with an RPATH into `build/bin` , so run `llama-server` in place. `-DLLAMA_OPENSSL=ON` is passed when the OpenSSL headers exist, and after the build llmctl reports whether the binary really has HTTPS support (CMake cache + linked `libssl`).
- `colibri` : builds the C engine targets (`colibri` , `qwen36` by default) via `make -C submodules/colibri/c <target>` , verifying each resulting binary is executable, then optionally `pip install -e submodules/colibri` to install the `coli` Python launcher (a clear `warn` , not a failure, if `pip` / `pip3` is unavailable).
- `onnx` : the `onnx` engine (`lib/onnx_server.py` , used by the `decide-nli` decision profile) is a pure-python3 runner with no compile step. `llmctl build onnx` creates a private venv (`LLMCTL_ONNX_VENV` , default `$LLMCTL_DATA_DIR/venv-onnx`) from the hash-locked requirements `lib/lock/requirements-onnx.lock` (never a system-wide `pip` ; needs Python >= 3.11 and PyPI access). Without the venv, the encoder runtime's guarded imports fail with a clear error and the post-download smoke test of `decide-nli` SKIPs with a recorded reason (the download is then NOT reported as fully verified).

- `decide`: builds the Go binary `build/llmctl-decide` from `cmd/llmctl-decide` (override the output path with `LLMCTL_DECIDE_BUILD_OUT`, the binary location with `LLMCTL_DECIDE_BIN`). Without Go the command says exactly what to run.

`LLMCTL_DRY_RUN=1` prints every `cmake` / `make` / `pip` command instead of running it.

install

```
llmctl install
```

Installs (or refreshes) the persistent-service backend for this OS (`svc_install`, dispatched via `sched_load_backend`):

- **Linux**: writes systemd `--user` unit templates `llmctl-llama@.service` and `llmctl-colibri@.service` into `${LLMCTL_UNIT_DIR:-$XDG_CONFIG_HOME/systemd/user}`. Each instance reads its launch parameters from an `EnvironmentFile` written by `enable` / `start`. Each profile gets its own memory drop-in: `MemoryHigh` = 1.25x and `MemoryMax` = 1.5x of the plan footprint (floors apply, always below 90% of RAM), with `MemorySwapMax=0` (see the comment in `lib/service_linux.sh`). The units use `Restart=on-failure` / `RestartSec=5` with `StartLimitBurst=3` in `StartLimitIntervalSec=900` and `RestartPreventExitStatus=SIGKILL SIGTERM 98` for crash-loop bounding. Runs `systemctl --user daemon-reload` and attempts `loginctl enable-linger` so services survive logout.
- **macOS**: prepares the LaunchAgents directory (`${LLMCTL_PLIST_DIR:-~/Library/LaunchAgents}`); per-profile plists are written individually by `enable` / `start`, not by `install`.

enable <profile>

```
llmctl enable coder
```

Writes the profile's launch environment (or plist, on macOS), enables autostart at login, **and** starts it now (`main()` 's `enable` branch in `bin/llmctl`): computes a fresh hardware plan, resolves the profile's mode/port/ctx/ngl/parallel/flash-attn/ram/vram from it, calls `sched_build_launch` , writes the env file, touches `$LLMCTL_SERVICES_DIR/<profile>.enabled` , calls `svc_enable` (which starts the service via `systemctl --user enable && start` or `launchctl bootstrap`), and records the scheduler reservation so budgeting stays correct. Prints `enabled and started <profile> (mode=<mode>, port=<port>)` .

An enabled profile is never evicted by `llmctl auto` 's LRU logic.

disable <profile>

```
llmctl disable coder
```

Reverses `enable` : stops and disables the service (`svc_disable`), then removes the `.enabled` marker and the runtime reservation file (`$LLMCTL_SERVICES_DIR/<profile>.enabled` , `$LLMCTL_RUNTIME_DIR/<profile>.run`).

start <profile> [more...]

```
llmctl start coder
llmctl start fast small     # co-resident: both together
```

Starts one or more profiles if their **combined** footprint fits the live RAM/VRAM budgets minus what is already reserved (`sched_start` → `_sched_start_impl` in `lib/scheduler.sh` , run under an exclusive

advisory lock so concurrent invocations never race). For each requested profile: an already-running profile is skipped with a log line; an unknown profile dies with `unknown profile: <p> (see: llmctl models list)`; a profile whose individual or cumulative footprint would exceed the budget aborts the *whole* start (nothing partially starts) with an error naming the exact shortfall, a suggested alternative profile that fits right now (if any), and a `llmctl switch <profile>` fallback.

On success each profile is launched via `sched_build_launch` + `svc_write_env` + `svc_start`, and a reservation record is written.

Real evidence (`tests/test_scheduler.sh`): `llmctl start fast small` prints `started fast (mode=<mode>, port=8080, reserved <N> MiB RAM + <N> MiB VRAM)` and the equivalent line for `small` (port 8085); a refused start prints `cannot start 'vision': needs <N> MiB RAM + <N> MiB VRAM, but only <N> MiB RAM + <N> MiB VRAM remain` followed by `suggested alternative that fits now: llmctl start <alt>`.

stop [profile|all]

```
llmctl stop coder     # stop one
llmctl stop           # stop everything (no args = all)
llmctl stop all       # stop everything (explicit)
```

Stops the named profile(s), or every running profile when called with no arguments or with `all` (`sched_stop` → `_sched_stop_impl`). Removes each stopped profile's runtime reservation file and prints `stopped <profile>`. Stopping does not disable autostart — an enabled profile stopped this way will restart on next login/boot unless also `disable`d.

restart <profile>

```
llmctl restart coder
```

Restarts one running service in place (`svc_restart` : `systemctl --user restart` on Linux, equivalent to `svc_start` / `kickstart -k` on macOS). Requires exactly one profile argument.

switch <profile>

```
llmctl switch vision
```

The always-works escape hatch: stops every running profile, then starts exactly the named one (`sched_switch` : `sched_stop all` followed by `sched_start <profile>`). Useful when a co-resident `start` would be refused for lack of budget.

Real evidence (`tests/test_scheduler.sh`): after `llmctl switch moe-fast` , the only profile reported by `sched_running` is `moe-fast` .

auto <capability> [...]

```
llmctl auto chat
llmctl auto coder vision
```

Resolves each requested capability (`chat` , `coder` , or `vision`) to the best-ranked catalog profile that is both recommended for this host and tagged with that capability (`sched_auto` → `_sched_auto_impl` in `lib/scheduler.sh`), using the fixed ranking in `sched_rank_for_capability` (e.g. for `chat` : `colibri-glm ws-dense-32b ws-moe-30b coder moe-fast fast vision-pro vision colibri-qwen36 small` , best first). If the resolved set does not currently fit the RAM/VRAM budget alongside what is already running, `auto`

evicts the least-recently-started **non-enabled** running service (LRU) — one at a time, up to 16 attempts — until it fits, printing `auto: evicting '<profile>' (LRU, not enabled) to make room` for each eviction. Enabled services and profiles already in the target set are never evicted; if nothing evictable remains, it fails with `cannot satisfy 'auto <caps>': remaining services are enabled (protected)` and suggests `llmctl disable <profile>` or `llmctl switch <profile>`.

Real evidence (`tests/test_scheduler.sh`): `llmctl auto vision` prints `evicting 'fast'` when eviction is required; `llmctl auto coder vision` prints `capability 'coder' -> profile '<name>'` for each capability resolved.

decide

Typed decisions (`noul` yes/no probability, `choice` one-of-N with probabilities, `score` probability-weighted level) against the local decision profiles (capability `decide` ; e.g. `decide-tiny` 8092, `decide` 8093, `decide-pro` 8094, `decide-nli` 8096 on the `onnx` engine, `decide-2b` 8098, `decide-max` 8097 — the current full table is in `docs/decision-models.md`). The logic is the **Go binary** **`llmctl-decide`** (`cmd/llmctl-decide` , built by `llmctl build decide`): an HTTPS gateway (`serve`), a client (`ask` , `batch` , `models`), access-key and certificate management (`key` , `cert`) and a one-question engine `smoke` . `lib/decide.sh` is a thin bash front end that delegates to it, plus the `capacity` / `status` reports and the interactive wizard. The profile's catalog `decision.protocol` selects the mechanism (`letter-logit` : shared lettered-option prompt + first-token logprob readout against `llama-server` ; `nli-onnx` : one premise/hypothesis pair per option scored by the internal encoder runtime `lib/onnx_server.py`). Concept, mechanism, and honest limitations: `docs/decision-models.md` ; gateway reference: `docs/decide-gateway.md` . Subcommands the shell front end (`cmd_decide`) accepts: `ask` , `batch` , `models` , `capacity` , `status` , `interactive` , `serve` , `key` , `cert` , `registry` , `port` , `discover` , `schema` , `smoke` , `mcp` , `vantage` and `help` ; the last three are forwarded verbatim to the

Go binary (in the first 3.1.0 candidate the front end answered `unknown decide subcommand` for them; that was fixed, and `tests/test_decide_cli.sh` compares the forwarded list with the commands the binary registers).

decide ask

```
llmctl decide ask --type {noul|choice|score} (--state S | --state-file F | --stdin)
    --instructions I [--criteria JSON | --criteria-file F] [--profile P] [--json]
```

Sends one typed question to the gateway over HTTPS (CA-verified; the access key is resolved by the client from `LLMCTL_API_KEY` or the installation `.env` and is never on a command line) and prints the typed answer as single-line JSON (pretty-printed only on a TTY without `--json`). The state travels as data (`--state-file` / `--stdin`), never as an argument. The answer carries the hosted shape plus `model` and `evidence: {profile, port, latency_ms}`.

Flags: `--profile` (default: the gateway's default profile); `--type` (required); `--state` / `--state-file` / `--stdin`; `--instructions` (required - the question); `--criteria` (required for `choice` / `score`, optional for `noul`); `--endpoint` / `--cacert`; `--min-confidence X` (withhold a low-confidence answer: exit 10, JSON still printed); `--explain` (print the prompt/pairs and the per-option probabilities); `--dry-run`; `--json`; `--interactive` (delegate to the wizard, see below); `--question-file F` (one Typed Question JSON object instead of `--type/--instructions/--criteria`); `--permute K` (2..64: ask K cyclic option orders of every choice question and print the order-averaged answer with `evidence.permute.flip_rate`); `--retries N` (extra attempts after HTTP 429/503/529) and `--timeout SEC`.

Criteria shapes:

- `noul` — optional `{"true": "desc of yes", "false": "desc of no"}`.
- `choice` — `{"option-key": "description", ...}`, 2..20 options (`LLMCTL_DECIDE_MAX_OPTIONS`, hard cap 26 letters A..Z).

- `score` — `["level 0 desc", "level 1 desc", ...]`, 2–10 ordered levels.

Examples:

```
llmctl decide ask --type noul --state "Arithmetic facts." \
    --instructions "Is 2+2=4?"
# {"model":"decide-tiny","answers":{"q":
        {"type":"noul","noul":0.97...}},"evidence":{...}}

llmctl decide ask --profile decide --type choice --state
        "Routing." \
    --instructions "Which team handles invoices?" \
    --criteria '{"billing":"handles
        invoices","legal":"contracts"}'
# {"model":"decide","answers":{"q":
        {"type":"choice","choice":"billing","probabilities":
        {...},"confidence":...}},...}

llmctl decide ask --type score --state "The deployment
        succeeded." \
    --instructions "Rate the quality of this outcome." \
    --criteria '["bad outcome","mixed outcome","excellent
        outcome"]'
```

Exit codes: 0 success; 1 backend/readout failure; 2 usage error (unknown type/flag, missing required flag, malformed criteria or numeric variable, option cap exceeded); 4 access-key problem; 5 certificate/TLS problem; 6 gateway not ready or unreachable; 10 abstained.

decide capacity

```
llmctl decide capacity [--json]
```

Renders the planner's `decision_instances` subtree: per decide profile, the max parallel instances in GPU mode and in CPU mode (**alternative placements, never additive**), slots per instance, per-instance RAM/VRAM, and tier-gate/fit reasons. This is a **read-only capacity report** — it reserves and starts nothing (v1 cannot launch N copies of one profile; `LLMCTL_PORT_<PROFILE>` gives one port override).

Example (the `tests/fixtures/hw-baseline.json` fixture — asserted exactly by `tests/test_decide.sh`):

```
Host tier: baseline   RAM budget 25904 MiB, VRAM budget 10444 MiB
profile        tier-ok  gpu-instances  cpu-instances  slots/instance  ram/instance
decide         yes      2              7              2               2048 MiB
decide-pro     no       0              0              2               2048 MiB
decide-tiny    yes      6              16             4               2048 MiB
```

(total_decision_slots: decide-tiny 64, decide 14, decide-pro 0.)

decide status

```
llmctl decide status [--json]
```

Table of every decide-capable catalog profile (`profile` , `port` , `running` = scheduler reservation exists, `enabled` = autostart marker exists), followed by the gateway (`llmctl-decide serve --status` , verified process identity) and the registry. `--json` prints `{"profiles": [...], "gateway": {"available", "running", "detail"}, "registry": ...}` ; when the binary is not built the gateway/registry parts are reported as unavailable, not as an error. Any other argument is exit 2.

decide interactive

```
llmctl decide interactive [--profile P] [--type T] [--state S]
    [--state-file F] [--instructions I] [--criteria JSON] [--interactive]
```

A seven-step wizard (profile → type → state → instructions → criteria → confirmation → run) that is a **thin prompter over `decide ask`** — it contains no decision logic; every step has a flag/env equivalent and any step given that way is never prompted for. All prompts go to stderr; only the final single-line JSON lands on stdout (the wizard stays machine-pipeable).

Interactive-mode precedence rule (the project's single sanctioned interactive exception): non-interactive is the default everywhere. The wizard activates only via: (1) the explicit `llmctl decide interactive` subcommand on a TTY; (2) the `--interactive` flag on `decide`

`interactive` or `decide ask` (which also allows piped stdin, e.g. scripted heredocs); or (3) bare `llmctl decide` when both stdin and stdout are TTYs. Non-TTY stdin without `--interactive` is rc 2 (`interactive mode requires a TTY`), and `LLMCTL_DECIDE_NO_INTERACTIVE=1` makes every interactive path rc 2 — set it in CI.

decide serve

```
llmctl decide serve [--foreground] [--status] [--stop] [--bind H] [--port N]
llmctl decide serve --enable [--now] | --disable
```

Runs the Go HTTPS gateway (Jev/TypeSafe-shaped `POST /v1/systemone` ; default port `LLMCTL_DECIDE_PORT` , 8095). It refuses to start without a valid access key (generated on first start into the installation `.env` , mode 0600) or certificate (created under `$LLMCTL_HOME/cert`). Default: detached, pidfile `$LLMCTL_STATE_DIR/decide/gateway.pid` ; `--foreground` execs in place (for systemd/launchd); `--status` is rc 0 only for a verified running gateway; `--stop` signals only a process verified to be this gateway (a pidfile naming an unrelated process is refused; no pidfile is a clean no-op). Every `/v1/*` request needs `Authorization: Bearer <key>` ; `/healthz` and `/readyz` stay open. Point a TypeSafe SDK at it with `TYPESAFE_BASE_URL=https://127.0.0.1:8095` , `TYPESAFE_API_KEY=$LLMCTL_API_KEY` and the local CA (`SSL_CERT_FILE`). Full reference: `docs/decide-gateway.md` .

`--enable` installs the gateway as a persistent boot-time user service (`llmctl-decide-gateway.service` on Linux, a launchd agent on macOS), enables it and starts it (`--now` is the explicit spelling of that default); `--disable` stops it, disables it and removes the unit and the gateway's registry row. Both are idempotent and take no other flag (set the port and bind in `$LLMCTL_STATE_DIR/decide/gateway.conf`). `--enable` refuses (rc 1) while an installed engine unit predates the current generator, naming it; the fix is `llmctl install` . To pick up a changed unit or wrapper: `systemctl --user restart llmctl-decide-gateway.service` . Persistent engines register at the path they really

serve (llama `/health`, onnx `/readyz` (truthful readiness; `/healthz` is liveness only), colibri `/v1/models`). Evidence: `tests/test_decide_serve_enable.sh` (systemctl stubbed, no real unit is created; macOS is a dry run only). The full service picture (linger, `gateway.conf`, drop-ins, CPU-adaptive timeout, the 15 s drain grace on stop, LAN binding) is in persistent-services and lan-exposure.

Real evidence (`tests/test_gateway_endpoints.sh`): the built binary serves HTTPS and answers every reachable row of the endpoint inventory (including `401` for an absent or wrong key), `serve --status` is 0 while running, `--stop` removes the pidfile and the port closes, and a pidfile naming an unrelated process is refused. `LLMCTL_DRY_RUN=1 llmctl decide serve ...` prints `DRY-RUN: <binary> serve <args>` and starts nothing (`tests/test_decide.sh`).

decide smoke

(`llmctl decide smoke ...` forwards to the binary; `llmctl models download` calls the binary itself.)

```
llmctl decide smoke --url URL --protocol letter-logit|nli-onnx|systemone-native
    [--key-file F] [--options N] [--expect-choice KEY] [--json]
```

Asks ONE engine a fixed deterministic choice question through the production driver for the protocol. Exit 0 only for a valid typed answer (finite probabilities summing to 1, and `--expect-choice` if given); 1 backend failure, 2 usage, 6 unreachable. The post-download decision smoke test of `llmctl models download` is built on it.

decide batch

```
llmctl decide batch [--in FILE|-] [--out FILE|-] [--profile P] [--min-confidence X]
```

Newline-delimited JSON in (`{"id":..., "state":..., "questions":{...}}`), newline-delimited JSON out in input order. A per-line failure is reported as `{"id":..., "error":{...}}` and the exit code is the highest-severity code seen.

decide models

`llmctl decide models [--json]` lists the models the gateway serves (profile ids and aliases such as `jev-latest`, protocol, status and the advertised `limits`: `max_options`, `score_levels`, `max_state_chars`, `max_context_tokens`, `max_pairs`). Needs the access key.

decide key

```
llmctl decide key {doctor|show --yes-print|path|rotate [--grace N]|export --file F |
```

`doctor` reports the key source (`env`, `file`, `none`), the file mode and shadowing, never the value. `show --yes-print` is the only command that prints the key. `path` prints the `.env` path. `rotate` writes a new key atomically, prints a `scope:` line and an update checklist (see runbooks for the environment-key caveat). `export` adds a managed block to a startup file (reference form by default). Exit 4 = key problem (including a weak operator-supplied key).

decide cert

```
llmctl decide cert {ensure [--mode ca-leaf|selfsigned|byo] [--san dns:N,ip:A] [--off
```

`ensure` is idempotent; `show` prints paths, SHA-256 fingerprints, SANs and expiry; `export` copies the public CA (0644); `renew` re-issues the leaf and never touches the CA; `doctor` checks key/cert match, expiry (warns within 30 days), SAN drift and permissions. Exit 5 = certificate problem. There is no `cert reload`: send `SIGHUP` to the gateway or restart it.

decide registry | port | discover

Service registry and port allocator (details: registry-discovery, recipes: runbooks): `registry {register|unregister|list|reconcile|ack-`

`corrupt|diff}` , `port {allocate|release|list}` , `discover [--json] [--kind K] [--label k=v] [--healthy-only]` .

decide schema

`llmctl decide schema [--format json-schema|openai-tool|mcp]` prints the typed-question schema as a tool definition for coding agents (no network, no key). The MCP server itself is `build/llmctl-decide mcp` (stdio, one tool `decide` ; a configuration problem becomes a tool error, never a default answer). See agents.

decide scale

`llmctl decide scale <profile> <N>` starts or stops instances of one decision profile until `N` run (OD-23; shell scheduler, `lib/scheduler.sh`). Instances are `<profile>` , `<profile>.2` , ...; scale-up is admission-bounded and all-or-nothing (exit 3 with the needed-versus-remaining MiB, nothing started), further instances get registry-allocated ports (`llmctl build decide` provides the allocator), scale-down stops the highest-numbered first.
`LLMCTL_DECIDE_MODE=deterministic|throughput` chooses how the gateway spreads requests. Details, exit codes and sample output: scripts/decide. Reporting only, without starting anything: `llmctl decide capacity` .

decide calibrate | probe-order | completions

Implemented in the `llmctl-decide` binary (operator decision OD-23); `llmctl decide <name>` forwards to them.
`calibrate --profile P --labels F [--method temperature|platt|isotonic]` reports accuracy with a Wilson interval, baselines and ECE/MCE/Brier and writes a profile bound to the model sha256 and the prompt-template hash (with fewer than 200 labels, isotonic with fewer than 1000, or all-same outcomes, only the report is printed: no profile is written at all). `probe-order --questions F [--permute K]` measures option-order sensitivity. `completions {bash|zsh}` prints a completion script. Flags, label-file format and exit codes: scripts/decide; how the

gateway applies a profile: decide-gateway; readable fields: calibration-tool-fields.

admit

`llmctl admit <hf-repo> [--paper-only] [--json]` runs the candidate-model admission gates G1-G10; `llmctl admit --all` and `--summarize` cover the research register. It is a catalog-maintainer tool; gate G10 (a real run) is pending for 13 candidates (limitations).

status

```
llmctl status
```

Human-readable table of every running profile (`sched_status`): `profile` , `port` , `mode` , `RAM MiB` , `VRAM MiB` , `enabled` (yes/no), and `state` . `state` is `running` , or `failed (crash-loop)` when `svc_is_failed` reports the service exceeded its restart bound (systemd's `StartLimitBurst` / `StartLimitIntervalSec` , or the launchd dry-run marker convention) — in which case the last line of the profile's log is printed immediately below its row, so a crash-looped service is never shown as just another healthy row. Prints `no llmctl services running` when nothing is up.

logs <profile> [lines]

```
llmctl logs coder        # last 100 lines (default)
llmctl logs coder 500    # last 500 lines
```

Tails `$LLMCTL_LOG_DIR/<profile>.log` (`svc_logs` , identical on both backends). Dies with `no log file yet: <path>` if the profile has never produced a log. `LLMCTL_DRY_RUN=1` prints the `tail` invocation instead of running it.

cluster | tenant | apikey (opt-in, need a running `llmctld`)

```
llmctl cluster join <peer-addr> | leave | status
llmctl tenant create <name> | list | quota <name> [--requests-per-second N] [--max-c
llmctl apikey create <scope> | rotate <key-id>
```

These are thin clients of the opt-in cluster daemon `llmctld` (design: cluster-architecture). Every subcommand first calls `cluster::require_daemon`; when the daemon cannot be reached the command fails and names the endpoint - llmctl never silently falls back to single-host scheduling. Single-host usage never touches them.

- **Transport (3.1.0).** The CLI talks to `${LLMCTL_CLUSTER_ENDPOINT:-https://127.0.0.1:9443}` with `curl` over **HTTP/3 with mutual TLS**, always verifying the daemon certificate (never `-k`). Daemon certificates carry SANs (127.0.0.1, ::1, localhost, the hostname, the `-api-bind` host and the new `-advertise` name). `llmctld cluster bootstrap|join` write `ca.crt`, `client.crt`, `client.key` into a 0700 directory (files 0600; default `cli/` next to the CA, `-cli-cert-dir` overrides) and print `CLI_CERT_DIR=<dir>`; `llmctld cluster issue-cli-cert` re-issues them. Configure the CLI with `LLMCTL_CLUSTER_CERT_DIR` (that directory), or per item `LLMCTL_CLUSTER_CACERT` / `LLMCTL_CLUSTER_CERT` / `LLMCTL_CLUSTER_KEY`, or `CURL_CA_BUNDLE` for the trust anchor only. A group/other-readable client key is refused. The bearer token (`LLMCTL_CLUSTER_TOKEN`) never appears on the command line (needs curl >= 7.55). Full detail: llmctld-cluster-tls.
- **The one remaining limit.** The host `curl` must list the `HTTP3` feature (`curl --version`); many distribution builds do not. Otherwise these commands report `llmctld unreachable` and the cluster test suites SKIP with a reason. How to get such a curl: llmctld-cluster-tls. Client certificates are valid 365 days and stop being trusted after a finalised CA rotation: re-run `llmctld cluster issue-cli-cert`.

- **What was verified.** The certificate/SAN/trust-bundle logic and the CLI argument construction are covered by tests, and the CLI was verified (2026-10-08) end to end with a real curl 8.22.0 built with ngtcp2/nghttp3 in a private test prefix: `cluster status`, `cluster join/leave`, `apikey` and `tenant` suites passed on the success path (loopback, one node). That curl is a private test build, not packaged by llmctl; the second-peer join scenario was not exercised.
- `cluster join` sends this host's `hostname` as the peer id and `<peer-addr>` as the address; `apikey create <scope>` uses the scope as both owner id and sole scope entry. Multi-node behaviour with a second real peer is covered by the Go integration tests of `llmctld`, not by a shell test.

version

```
llmctl version
```

Prints `llmctl <version>`, e.g. `llmctl 3.3.0` (the `LLMCTL_VERSION` constant at the top of `bin/llmctl`, kept equal to the `VERSION` file at the repository root). Real evidence (`tests/test_cli.sh`): the output contains the literal substring `llmctl 3.3.0`.

help

```
llmctl help
llmctl -h
llmctl --help
llmctl                # no command = help
```

Prints the full usage text (the `usage()` function in `bin/llmctl`, the same text `llmctl` prints when invoked with no arguments or an unrecognized command — an unrecognized command prints `unknown command: <cmd>` to stderr, then the usage text, then exits 2).

Constitution role and integrations

- **Hub engine.** llmctl is a constitution hub engine at `constitution/submodules/llmctl` (Constitution §11.4.28(C), §11.4.31): consumed by reference, never nested copies; `constitution`, `containers` and `superspec` are found by `lib/resolve_deps.sh` and linked by `make deps` (dependency layout).
- **§11.4.277.** `validate`, `vision validate`, `validate-all` and `require-validated` implement the validation gate. Decision-bearing use runs with `LLMCTL_VALIDATE_STRICT=1` and gates on `llmctl require-validated <profile>`; `0` pass, `1` wrong answer, `2` could not determine (`75` pressure refusal is reported as `2`; `98` port in use on the unit path). Diagram: validation.
- **§11.4.278.** The `vision` and `vision-pro` profiles are the Vision tiers a HelixQA consumer declares; verdicts must carry the model identity (validation).
- **§11.4.279.** The `decide` command family and the HTTPS gateway (bearer key mandatory) are llmctl's side of the decisions-model layer (decision models, decide gateway).
- **claude_toolkit (JEV).** claude_toolkit detects the local decide gateway and exposes it to agents as a decision layer (permission gate, MCP tool `jev_ask`, launch environment); llmctl does not depend on it. Diagram index: diagrams.

Environment variables

These affect command behavior across multiple commands rather than being specific to one; each is documented at its point of use in `usage()` and in the relevant `lib/*.sh` module.

Variable	Effect
`LLMCTL_DRY_RUN=1`	Every OS service action (`systemctl` / `launchctl` / `tail` / `curl` downloads/ `cmake` / `make`

Variable	Effect
	builds) is printed instead of executed. Env/unit/plist files are still written for real so their content is testable.
`LLMCTL_FAKE_HW=<file.json>`	Use a hardware fixture instead of a live probe (testing).
`LLMCTL_MODELS_DIR=<dir>`	Override where downloaded model files are stored.
`LLMCTL_CLUSTER_ENDPOINT=<url>`	`llmctld` API endpoint for `cluster` / `tenant` / `apikey` commands. Default `https://127.0.0.1:9443`.
`LLMCTL_CLUSTER_TOKEN=<jwt>`	Bearer token sent as `Authorization: Bearer <jwt>` on cluster/tenant/apikey HTTP requests.
`LLMCTL_SEED=<n>`	Appends `--seed <n> --temp 0` to a `llama` profile's launch arguments (deterministic live-challenge mode). Opt-in only; unset by default so everyday interactive sessions are unaffected.
`LLMCTL_SMOKE=0`	Disables the post-download smoke test in `models download` (still logged as an explicit skip, never silently omitted).
`LLMCTL_SMOKE_PORT` / `LLMCTL_SMOKE_TIMEOUT`	Port (default `auto` = a free ephemeral port per smoke test; a number pins it and is refused when in use) and timeout in seconds (default `120`) for the post-download smoke tests (GGUF, decision, onnx). Readi-

Variable	Effect
	ness is proven against the launched process, never just "something answers".
`LLMCTL_LLAMA_SERVER`	Override path to the `llama-server` binary (used by `start` / `enable` / `models download` 's smoke test).
`LLMCTL_COLI_BIN`	Override the `coli` launcher binary name/path used by `colibri` profile launches.
`LLMCTL_UNIT_DIR` (Linux)	Override the systemd `--user` unit directory `install` writes to.
`LLMCTL_PLIST_DIR` (macOS)	Override the LaunchAgents directory `install` / `enable` writes to.
`LLMCTL_DECIDE_PROFILE`	Default profile for `decide` commands (default: the first servable profile of the decide ranking that is downloaded (`decide-nli` , then `decide-2b` , `decide` , …); `decide-tiny` is catalogued but not servable by the gateway yet).
`LLMCTL_DECIDE_PORT`	Gateway port for `decide serve` (default `8095`).
`LLMCTL_DECIDE_TEMPERATURE`	Calibration temperature dividing letter logprobs before renormalization in the gateway's `letter-logit` driver (default `1.0`).
`LLMCTL_DECIDE_NO_INTERACTIVE=1`	Makes every `decide` interactive path exit 2 — set it in CI.
`LLMCTL_DECIDE_MAX_OPTIONS`	Practical cap on choice options (default `20` ; hard cap 26).

Variable	Effect
`LLMCTL_DECIDE_TIMEOUT`	Seconds: the gateway's end-to-end budget per request (default `8`; when unset and a CPU-placed llama decision instance (offload 0; the onnx encoder does not count) is running or enabled, the launcher (`svc_hook.sh run-gateway` / `decide serve`) raises it to `120` and the start-up banner says `cpu-adaptive`; an explicit value always wins, see `docs/decide-gateway.md`) and, for `decide ask`, the per-attempt wait (default `30`; `--timeout` overrides).
`LLMCTL_API_KEY`	The one access key of the decision gateway and llmctl's own clients (environment, else installation `.env`, else generated on first `decide serve`). Never printed except by `decide key show --yes-print`.
`LLMCTL_DECIDE_MAX_STATE_CHARS`	State budget for decoder profiles (default `8192`); over-budget is rejected with 422 unless `LLMCTL_DECIDE_TRUNCATE=1` (then head+tail shortened, header `x-llmctl-decide-truncated: true`).
`LLMCTL_ONNX_MAX_PAIRS` / `LLMCTL_ONNX_MAX_BODY_BYTES` / `LLMCTL_ONNX_MAX_CONCURRENCY` / `LLMCTL_ONNX_SOCKET_TIMEOUT`	Limits of the internal encoder scoring runtime (`lib/onnx_server.py`; defaults 64 / 4 MiB / 4 / 30 s). (Historical: the retired `LLMCTL_ONNX_FAKE` seam no longer exists.) the runtime key comes

Variable	Effect
	from a 0600 `--api-key-file`, never the environment.